

Project Document for Flock Simulation

Title: Flock Simulation Program

Student's Name: Triet

Student Number: 102703797

Degree Program: Data Science

Year of Studies: First Year

Date: 28.04.2025

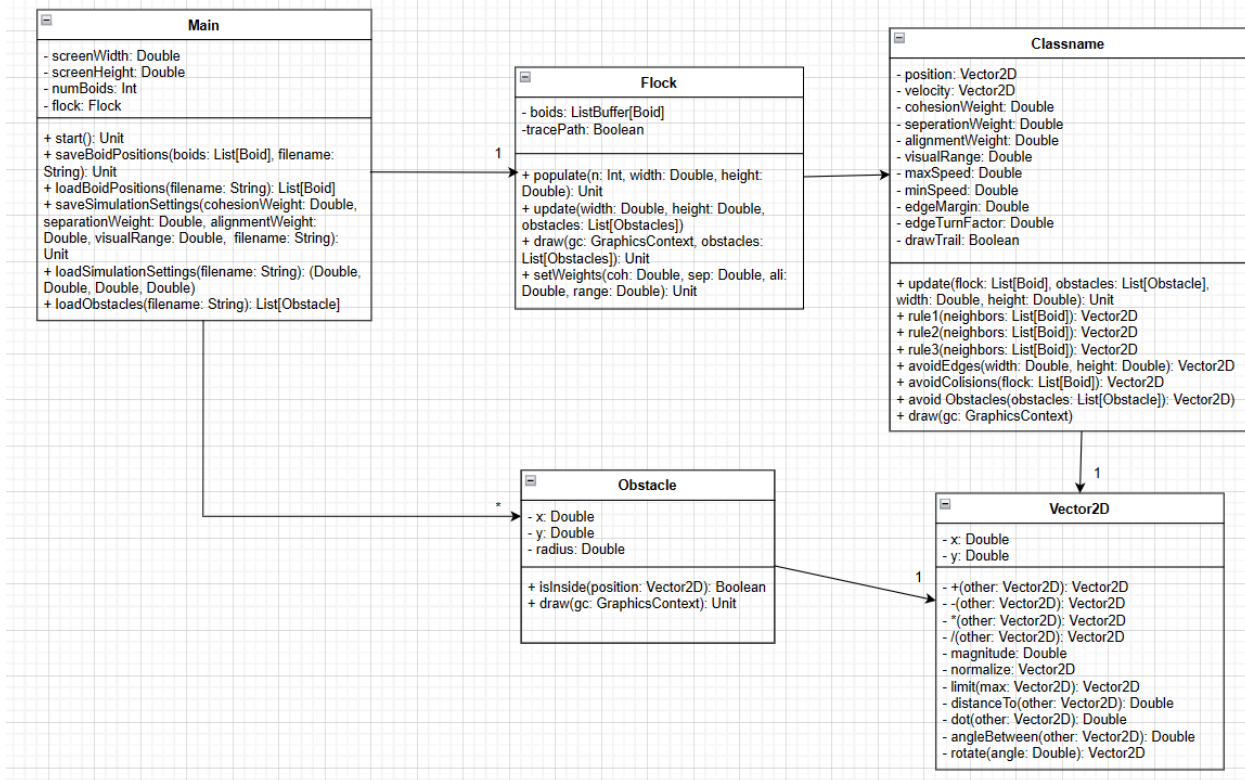
General Description

This program simulates the movement of birds in a 2D movement using Craig Reynold's "boids" model. Each bird in the simulation will follow three rules that will indicate its' movements: Separation(avoiding collisions with other birds), Alignment(matching the speed and direction with other birds), and Cohesion(moving towards the average position of the birds). Users will be able to specify the number of boids and adjust the weights of the three rules to see how different settings will affect flock behavior. The program will be able to display these movements in a 2D environment where users can observe the flock's behaviors in real-time.

User interface

Users can run the program from "Main", then a screen will show up with a flock of boids flying in random directions. The positions, velocities, and number of boids can be edited through the "boids_position.txt" file(writing in x,y coordinates and x,y velocities, respectively), and if the file is empty, 100 boids will be placed randomly. There are sliders at the bottom of the program for users to adjust the weight of the rules, the visual range of boids, and also a tick box for if the user want to trace the paths of the boids. Users can add obstacles to the screen through the "obstacles.txt" file(writing in the x,y coordinates and radius, respectively). If the file is empty, obstacles will not appear. After each simulation, the file will save the boids' and obstacles' positions, velocities, and it will load the same thing if users run it again.

Program structure



Comparing to the UML model in the technical plan, there are some very clear additions such as the additional class Vector2D. All the classes also had to include many more functions than I anticipated.

Algorithms

Find neighbours: Identify all characters within the defined distance and in the field of view angle. This is used to find neighbours in all of the three rules.

I. Cohesion Algorithm

Goal: Move toward the group's center.

$\text{cohesionVec} = (\sum \text{neighbor positions} / \text{size} - \text{boid position}).\text{normalize} * \text{cohesionWeight}$

II. Separation Algorithm

Goal: Avoid crowding nearby characters.

First filter those whose distance are < 20 from the boid, call them neighbors.

$\text{separationVec} = (\text{neighbor.position} - \text{position}).\text{normalize} / (\sum \text{distance of neighbors from the boid}).\text{normalize}$

III. Alignment Algorithm

Goal: Match the group's velocity/direction.

$\text{alignmentVec} = (\sum \text{neighbor velocities} / \text{size} - \text{boid velocity}).\text{normalize} * \text{alignmentWeight}$

IV. Edge avoidance Algorithm

Goal: Boids will turn to avoid the borders of the screen

$\text{steer} = \text{edgeTurnFactor}$ (if near top, left edge)

$\text{steer} = \text{edgeTurnFactor}$ (if near bottom, right edge)

edgeTurnFactor is a constant that determines how strongly the boid will steer away from the edge.

V. Collision avoidance Algorithm

Goal: Boids avoid other boids that are too close to prevent collisions.

$\text{distance} = (\text{neighbor.position} - \text{position}).\text{magnitude}$ (distance to other birds)

$\text{predicted position} = \text{position} + \text{velocity}$

$\text{directionAway} = (\text{predicted position} - \text{position}).\text{normalize}$

if $\text{distance} < 20$ $\text{avoidVec} += \text{directionAway} * (1.0 / \text{distance})$

VI. Obstacles avoidance Algorithm

Goal: Boids avoid obstacles

$\text{directionAway} = (\text{position} - \text{vector of x,y coordinates of obstacles}).\text{normalize}$

$\text{steer} += \text{directionAway} * \text{turn factor}$

Data structures

- `ListBuffer[Boid]` for Boid storage: Since the number of birds can change dynamically so `ListBuffer` is a suitable choice.

- List[Obstacle] was chosen because obstacles are static, so using an immutable list is efficient.
 - Vector2D is a custom class to manipulate Boids' positions and velocities, which involve two-dimensional movement so using a Vector2D is suitable.
-

Files

Boid Positions File (boids_positions.txt):

- Format: Plain text, each line contains 4 floating points for boids' position and velocities (x,y position, vx,vy velocity respectively)
- Example: 368.4559198883084 554.2847035804119 -0.5466064816833924 -0.8697574534856822
- Purpose: Save and load boids's positions and velocities

Simulation Settings File (simulation_settings.txt):

- Format: Plain text, each line contains the weight of each rule

```
cohesionWeight = 0.01  
separationWeight = 0.1  
alignmentWeight = 0.05
```

- Example: visualRange = 100.0
- Purpose: Save and load the simulations settings

Obstacles(obstacles.txt):

- Format: Plain text, each line contains x,y position and radius

```
100 200 30  
400 500 50
```

- Example: 600 100 40
 - Purpose: Use to load position into the simulation if users want to add anything
-

Testing

- System Testing:

+ User Interface Testing: Ensure the UI updates dynamically when adjusting weights or adding obstacles.

+ File Handling: Test loading and saving states with valid and invalid files.

+ Edge Cases:

- Birds near screen edges
- Flock behavior when obstacles are nearby
- Extreme weights of rules

- Unit Testing:

+ Core methods to test: All the rules in class Boid

Known bugs and missing features

The saving of simulations settings isn't working and the boids' movements when facing edges and obstacles could be refined to be more natural.

3 best sides and 3 weaknesses

3 best sides:

- The rules of boids works quite well
- The simulation allows users to twitch with the rules through the GUI
- There are additional features such as tracing boids' paths and also their visual range.

3 weaknesses:

- The commit history is a bit lacking because I forgot to connect my project to the repository. Once I did it, I was mainly done with the code so there was not much to see.
 - The boids movement when facing edges are a bit unnatural and could be refined.
 - Obstacles have to be added through the text file which can be a bit inconvenient, could be improved by letting users manipulate through the GUI.
-

Deviations

Due to procrastination, I did not start the project until the third and fourth week which pushes everything a bit behind. As a result, I couldn't refine the project to my liking and there are still some weaknesses in the program. I had to combine the File I/O and adding obstacles in the same week which was quite challenging too. In the future, if I were to do a project again, I would focus more on starting early and following the plan to better refine the program and stay more organized. Also, reading requirements and going through all the topics carefully on the A+ websites would help to avoid mistakes.

Final Evaluation

Overall, in terms of the program itself, I would say that it was good and met all the basic requirements. The weaknesses and strengths of the program I have all mentioned above and also necessary improvements as well. The programs are clearly separated into classes and I would say that it is easy to maintain and extend in the future. What I would do differently is mentioned in the deviations part.

References

This was the two link I used and was also mentioned in the technical plan.

- Craig Reynolds' article "Steering behaviors for autonomous characters"

<http://www.red3d.com/cwr/steer/gdc99/>

- Craig Reynolds' article "Boids" <http://www.red3d.com/cwr/boids/>